

025-进程调度

调度层次

级别	定义	状态转换
高级调度	决定是否加入执行的线程池，是否允许用户在终端连接	新建态 -> （挂起）就绪态 运行态 -> 终止态
中级调度	决定加入主存/ SWAP	三态和挂起相关状态间的转换
低级调度	决定哪个可用进程占用处理器执行	三态间转换

- 低级调度的主要功能
 - 记住进程或内核级线程的状态
 - 决定某个进程或内核级线程什么时候获得处理器，以及占用多长时间
 - 把处理器分配给进程或内核级线程
 - 收回处理器

处理器调度算法

- 资源利用率
- 响应时间：尽快处理实时任务
- 周转时间：提交到系统到执行完成的时间
- 吞吐量
- 公平性

优先数调度算法

- 给每个进程/作业规定优先数，先满足高优先数的进程/作业
- 非抢占式：等到当前进程执行完，CPU 空闲后，才调度下一个进程
 - 无 运行态 -> 就绪态的转换
 - 缺点：无法处理实时任务；任务独占 CPU，导致低优先级进程饥饿
 - FCFS 先来先服务
 - 将进程加入就绪队列，优先执行在就绪队列中时间最长的进程
 - 偏袒长进程，短进程饥饿
 - 多用于高级调度
 - SPN 最短进程优先
 - 偏袒短进程，长进程饥饿
 - HRRN 响应比高者优先
 - $响应比 = \frac{等待时间 + 估计服务时间}{估计服务时间}$
 - 防止长时间等待的饥饿现象
- 抢占式：若出现更高优先级的进程，则抢占当前进程
 - SRT 最短剩余时间优先
 - 抢占式：进程结束执行/新进程加入队列后都会触发调度

- 选择剩余时间最短的进程执行
- RR 时间片轮转调度算法
 - 每个进程有占用时间上限，超过时间片需让出 CPU
 - CPU 在时间片到达时产生中断，将正在执行的进程放回就绪队列（该进程的入队时间变成当前时间），基于 **FCFS** 调度下一个进程
 - 时间片长度
 - 过短：切换开销大
 - 过长：退化为先来先服务算法
 - 单时间片：所有时间片长度相同
 - 多时间片：不同时间片长度
 - 动态时间片：根据进程的优先级动态调整时间片长度
 - Feedback 分级调度算法/多级反馈调度
 - 根据优先级建立多级队列，高优先级队列时间片短
 - 步骤
 - 进程第一次进入就绪队列时，分配高优先级队列
 - 当被抢占，返回就绪态后，降低一级优先级
 - 基于 **FCFS** 和队列优先级调度下一个进程
 - 短进程会在高优先级队列中很快执行

彩票调度算法

- 随机抽取进程
- 合作进程间的彩票交换：动态调整彩票权重

现代 Linux 的调度器：Completely Fair Scheduler

由 GPT 补充

Prompt: 这个 PPT 太过时了，请为我描述一下现代 Linux 的调度原理

- 基于红黑树维护就绪进程，按虚拟运行时间（vruntime）排序。
- 不使用固定优先级队列和时间片轮转，而是追求 CPU 时间的“公平”分配。
- 每个进程都有 nice 值（权重），影响它获得 CPU 时间的比例。
- 调度选择红黑树中 vruntime 最小的进程运行。
- 时间片不是固定的，而是根据进程权重和总进程数动态计算。

题目注意

- 同时支持 x 道程序指的是多少个程序能进入可以抢占的队列中，一次还是只能执行一个
- 作业周转时间=完成时间-提交时间（不是开始执行时间）
- 时间片轮转中若进程提前结束，则不需要等待时间片到达，直接切换